

12th project

“Restaurant Kitchen and Dining Simulation”

Submitted to: Eng. Sarah Mohamed

Submitted by: Habiba Abouaya

- **Import Statements**

```
import simpy  
  
import random
```

- **Global Variables**

```
# Stores all simulation results and event data.  
  
wait_table_times = [] #stores each customer's waiting time to get a table  
  
wait_food_times = [] #stores waiting time for food preparation  
  
total_times = [] #total time each customer spends in the restaurant  
  
table_occupancy_times = [] #how long each table is occupied  
  
chef_usage_times = [] #how long chefs are actively cooking (for utilization)  
  
waitstaff_usage_times = [] #how long waitstaff are busy  
  
event_log = [] #stores every event (arrival, seating, serving, payment, etc.) for building  
the simulation table
```

- **Function that logs each event with time, queues, busy resources, and waiting time.**

```
def log_event(env, name, event, restaurant, wait_val=None):  
    event_log.append((  
  
        env.now, name, event,  
  
        len(restaurant.tables.queue),  
  
        len(restaurant.chefs.queue),
```

```

len(restaurant.waitstaff.queue),

len(restaurant.tables.users),

len(restaurant.chefs.users),

len(restaurant.waitstaff.users),

    wait_val
))

```

- **This class defines all the resources of the restaurant.**

```

class Restaurant:

    def __init__(self, env, num_tables, num_chefs, num_wait, num_stations):

        self.tables = simpy.Resource(env, num_tables)

        self.chefs = simpy.Resource(env, num_chefs)

        self.waitstaff = simpy.Resource(env, num_wait)

        self.stations = simpy.Resource(env, num_stations)

```

- **Function that generates customers according to an exponential inter-arrival process.**

```

def customer_arrival(env, restaurant):

    customer_id = 0

    while True:

        yield env.timeout(random.expovariate(1/8))

        customer_id += 1

        env.process(customer(env, f"Customer {customer_id}", restaurant))

```

- **Process represents the complete customer journey from arrival to exit.**

- **Customer flow**

- 1. Arrival → exponential interarrival time**
- 2. Seating → waits if no table is free**
- 3. Order taking → requires waitstaff**
- 4. Cooking → requires both a chef and a cooking station**
- 5. Serving → requires waitstaff**
- 6. Dining → customer eats**
- 7. Payment → requires waitstaff**
- 8. Departure → customer leaves system**

#Tracks when the customer arrives and logs the event

```
def customer(env, name, restaurant):
```

```
    arrival = env.now
```

```
    log_event(env, name, "Arrived", restaurant)
```

#Customer waits for a table, logs waiting time, and simulates the seating duration

```
    log_event(env, name, "Requested Table", restaurant)
```

```
    seat_req = env.now
```

```
    with restaurant.tables.request() as req:
```

```
        yield req
```

```
    wait_table = env.now - seat_req
```

```
    wait_table_times.append(wait_table)
```

```
    seat_time = env.now
```

```
log_event(env, name, "Seated", restaurant, wait_table)
```

```
yield env.timeout(random.uniform(1, 3))
```

#Customer requests a waiter, order is taken, and waitstaff busy time is recorded

```
log_event(env, name, "Order Taking Requested", restaurant)
```

```
with restaurant.waitstaff.request() as req:
```

```
    yield req
```

```
    ws_start = env.now
```

```
    log_event(env, name, "Order Taking Started", restaurant)
```

```
    yield env.timeout(random.uniform(2, 4))
```

```
    ws_end = env.now
```

```
    waitstaff_usage_times.append(ws_end - ws_start)
```

```
    log_event(env, name, "Order Taken", restaurant)
```

#Customer waits for a chef and station. Cooking time and chef utilization are recorded

```
log_event(env, name, "Cooking Requested", restaurant)
```

```
cook_req = env.now
```

```
with restaurant.chefs.request() as chef, restaurant.stations.request() as station:
```

```
    yield chef & station
```

```
    wait_food = env.now - cook_req
```

```
    wait_food_times.append(wait_food)
```

```
    chef_start = env.now
```

```
    log_event(env, name, "Cooking Started", restaurant, wait_food)
```

```
yield env.timeout(abs(random.normalvariate(10, 2)))

chef_end = env.now

chef_usage_times.append(chef_end - chef_start)

log_event(env, name, "Cooking Finished", restaurant)
```

#Waitstaff delivers the food and serving time is simulated

```
log_event(env, name, "Serving Requested", restaurant)
with restaurant.waitstaff.request() as req:
```

```
    yield req

    ws_start = env.now

    log_event(env, name, "Serving Started", restaurant)

    yield env.timeout(random.uniform(1, 3))

    ws_end = env.now

    waitstaff_usage_times.append(ws_end - ws_start)

    log_event(env, name, "Served", restaurant)
```

#Customer eats their meal

```
log_event(env, name, "Dining Started", restaurant)

yield env.timeout(abs(random.normalvariate(30, 10)))

log_event(env, name, "Dining Finished", restaurant)
```

#Payment is processed by waitstaff; waitstaff busy time is added

```
log_event(env, name, "Payment Requested", restaurant)
```

```
with restaurant.waitstaff.request() as req:
```

```
    yield req
```

```
    ws_start = env.now
```

```
    log_event(env, name, "Payment Started", restaurant)
```

```
    yield env.timeout(random.uniform(2, 5))
```

```
    ws_end = env.now
```

```
    waitstaff_usage_times.append(ws_end - ws_start)
```

```
    log_event(env, name, "Payment Complete", restaurant)
```

```
#Records total time spent and table occupancy duration
```

```
total = env.now - arrival
```

```
total_times.append(total)
```

```
table_occ = env.now - seat_time
```

```
table_occupancy_times.append(table_occ)
```

```
log_event(env, name, "Left", restaurant, total)
```

- **Function runs the entire simulation, resets lists, and computes statistics**

```
def run_sim(sim_time=600, seed=None):
```

```
    global wait_table_times, wait_food_times, total_times, table_occupancy_times
```

```
    global chef_usage_times, waitstaff_usage_times, event_log
```

```
    wait_table_times = []
```

```
    wait_food_times = []
```

```

total_times = []

table_occupancy_times = []

chef_usage_times = []

waitstaff_usage_times = []

event_log = []

if seed:

    random.seed(seed)

env = simpy.Environment()

restaurant = Restaurant(env, 20, 4, 6, 4)

env.process(customer_arrival(env, restaurant))

env.run(until=sim_time)

def avg(lst):

    return sum(lst) / len(lst) if len(lst) else 0

result = (

    len(total_times),

    avg(wait_table_times),

    avg(wait_food_times),

    avg(total_times),

    avg(table_occupancy_times),

    sum(chef_usage_times) / (4 * sim_time),

```



```
        sum(waitstaff_usage_times) / (6 * sim_time)
    )
```

```
    return result
```

- **Runs the simulation and stores the output statistics**

```
results = run_sim()
```

- **Extracts simulation results and prints key performance metrics**

```
(total_customers,
```

```
    avg_wait_table,
```

```
    avg_wait_food,
```

```
    avg_total_time,
```

```
    avg_table_occ,
```

```
    chef_util,
```

```
    waitstaff_util) = results
```

```
print("\n--- Simulation Results ---")
```

```
print(f"Total Customers Served: {total_customers}")
```

```
print(f"Average Wait for Table: {avg_wait_table:.2f} min")
```

```
print(f"Average Wait for Food: {avg_wait_food:.2f} min")
```

```
print(f"Average Total Time: {avg_total_time:.2f} min")
```

```
print(f"Average Table Occupancy: {avg_table_occ:.2f} min")
```

```
print("\n--- Staff Utilization ---")
```

```
print(f'Chef Utilization: {chef_util:.3f} ({chef_util*100:.1f}%)')
```

```
print(f'Waitstaff Utilization: {waitstaff_util:.3f} ({waitstaff_util*100:.1f}%)')
```

#The output

```
--- Simulation Results ---
Total Customers Served: 84
Average Wait for Table: 0.00 min
Average Wait for Food: 0.01 min
Average Total Time: 49.84 min
Average Table Occupancy: 49.84 min

--- Staff Utilization ---
Chef Utilization: 0.354 (35.4%)
Waitstaff Utilization: 0.200 (20.0%)
```

- **Simulation table printing**

```
print("\n--- Simulation Table (First 25 Events) ---")
```

```
header = f'{"Time":<7} {"Customer":<12} {"Event":<25} {"TblQ":<5} {"ChefQ":<5}
```

```
 {"WaitQ":<5} {"TblBusy":<8} {"ChefBusy":<8} {"WaitBusy":<9} {"Wait":<6}"
```

```
print(header)
```

```
print("-" * len(header))
```

```
for e in event_log[:25]:
```

```
    print(f'{e[0]:<7.2f} "
```

```
        f'{e[1]:<12} "
```

```
        f'{e[2]:<25} "
```

```
        f'{e[3]:<5} "
```

```
        f'{e[4]:<5} "
```

```
        f'{e[5]:<5} "
```

```
        f'{e[6]:<8} "
```

```
        f'{e[7]:<8} "
```

f'{e[8]:<9} "

f'{'-' if e[9] is None else f'{e[9]:.2f}':<6}')

--- Simulation Table (First 25 Events) ---

Time	Customer	Event	TblQ	ChefQ	WaitQ	TblBusy	ChefBusy	WaitBusy	Wait
5.97	Customer 1	Arrived	0	0	0	0	0	0	-
5.97	Customer 1	Requested Table	0	0	0	0	0	0	-
5.97	Customer 1	Seated	0	0	0	1	0	0	0.00
7.59	Customer 1	Order Taking Requested	0	0	0	0	0	0	-
7.59	Customer 1	Order Taking Started	0	0	0	0	0	1	-
9.99	Customer 1	Order Taken	0	0	0	0	0	1	-
9.99	Customer 1	Cooking Requested	0	0	0	0	0	0	-
9.99	Customer 1	Cooking Started	0	0	0	0	1	0	0.00
14.23	Customer 2	Arrived	0	0	0	0	1	0	-
14.23	Customer 2	Requested Table	0	0	0	0	1	0	-
14.23	Customer 2	Seated	0	0	0	1	1	0	0.00
15.25	Customer 3	Arrived	0	0	0	1	1	0	-
15.25	Customer 3	Requested Table	0	0	0	1	1	0	-
15.25	Customer 3	Seated	0	0	0	2	1	0	0.00
15.76	Customer 2	Order Taking Requested	0	0	0	1	1	0	-
15.76	Customer 2	Order Taking Started	0	0	0	1	1	1	-
16.39	Customer 4	Arrived	0	0	0	1	1	1	-
16.39	Customer 4	Requested Table	0	0	0	1	1	1	-
16.39	Customer 4	Seated	0	0	0	2	1	1	0.00
17.34	Customer 1	Cooking Finished	0	0	0	2	1	1	-
17.34	Customer 1	Serving Requested	0	0	0	2	0	1	-
17.34	Customer 1	Serving Started	0	0	0	2	0	2	-
17.88	Customer 3	Order Taking Requested	0	0	0	1	0	2	-
17.88	Customer 3	Order Taking Started	0	0	0	1	0	3	-
18.60	Customer 4	Order Taking Requested	0	0	0	0	0	3	-